

# Lecture 1: Introduction to diffusion models

---

**Daniel Pfrommer**

**January 5, 2026**



# Outline

---

1. Why use generative models?
2. A brief history of generative models:

**VAEs → GANs → Diffusion → ???**

3. Course logistics
4. What are diffusion models?

# Classical Supervised Learning

---

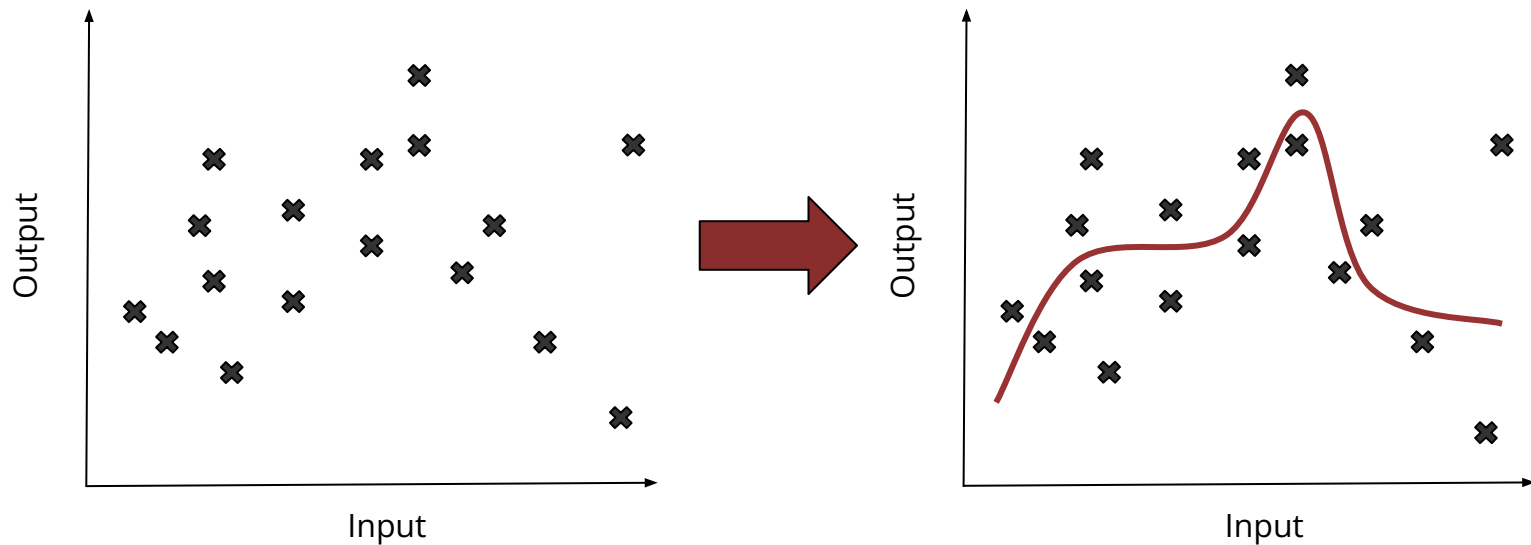


Image credit: SDXL: Improving Latent Diffusion Models for High-Resolution Image Synthesis <https://arxiv.org/pdf/2307.01952>



Massachusetts Institute of Technology

6.S183 IAP 2026: A practical introduction to diffusion models

# Generative Learning

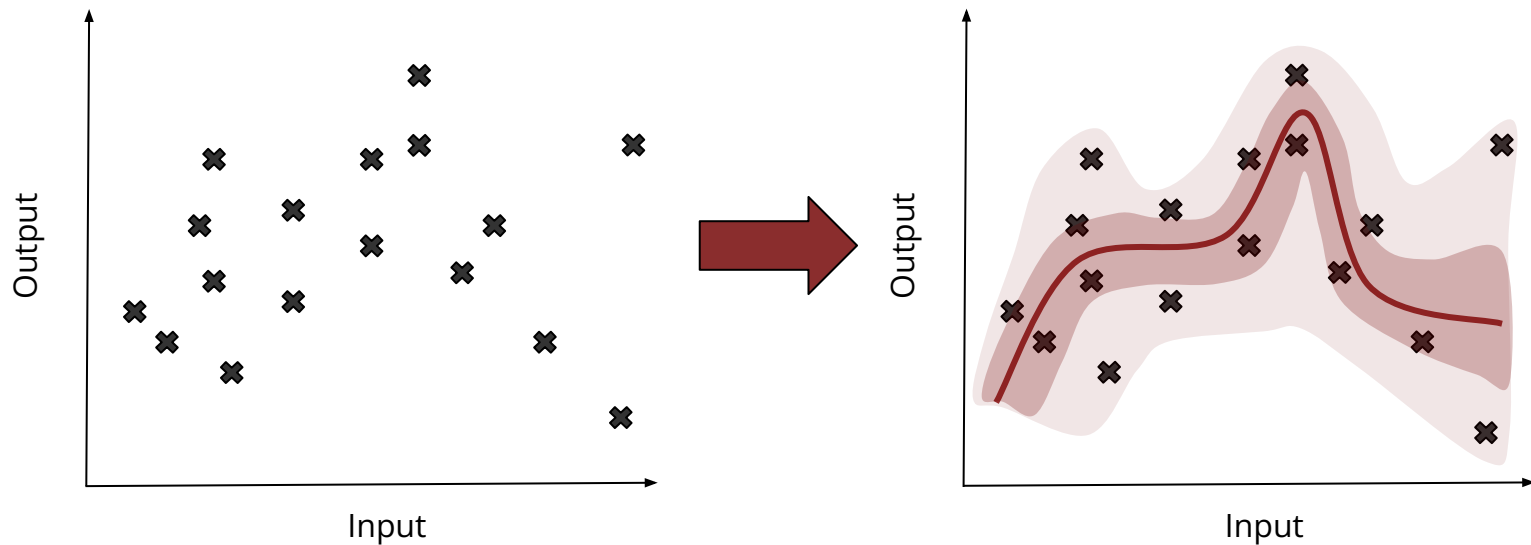


Image credit: SDXL: Improving Latent Diffusion Models for High-Resolution Image Synthesis <https://arxiv.org/pdf/2307.01952>

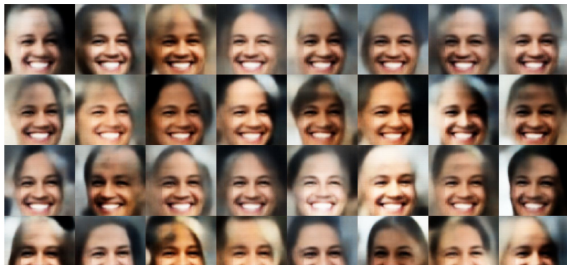


Massachusetts Institute of Technology

6.S183 IAP 2026: A practical introduction to diffusion models

# Many-to-one: Text-to-image generation

smile



wearing\_glasses

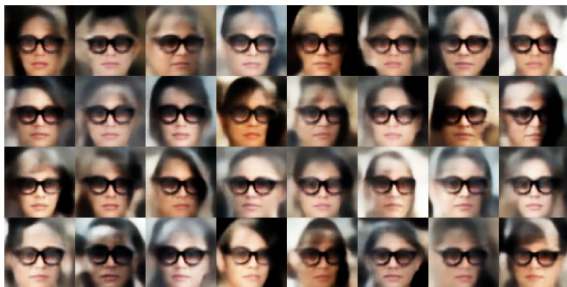


Image credit: [https://github.com/meanna/CVAE\\_CONV](https://github.com/meanna/CVAE_CONV)

## Regression-based Model



Image credit: SDXL: Improving Latent Diffusion Models for High-Resolution Image Synthesis <https://arxiv.org/pdf/2307.01952>

## Latent Diffusion Model



Massachusetts Institute of Technology

6.S183 IAP 2026: A practical introduction to diffusion models

# Sketch-to-Image: coarse-grained control

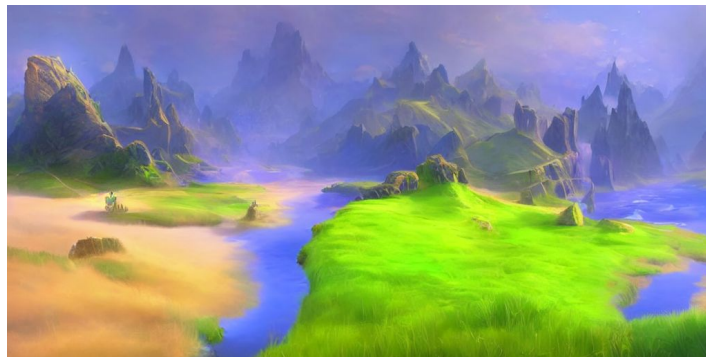
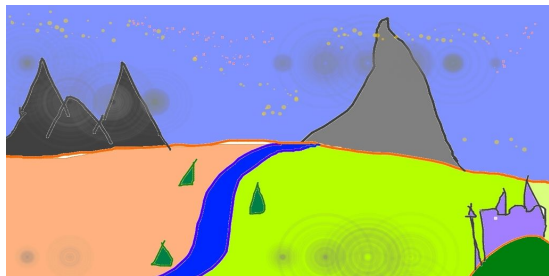


Image credit: <https://github.com/CompVis/stable-diffusion>



# Image editing and composition



Figure 1. With just a few images (typically 3-5) of a subject (left), *DreamBooth*—our AI-powered photo booth—can generate a myriad of images of the subject in different contexts (right), using the guidance of a text prompt. The results exhibit natural interactions with the environment, as well as novel articulations and variation in lighting conditions, all while maintaining high fidelity to the key visual features of the subject.

Image credit: DreamBooth: Fine Tuning Text-to-Image Diffusion Models for Subject-Driven Generation <https://arxiv.org/pdf/2208.12242>



# Diffusion “prior” for image restoration

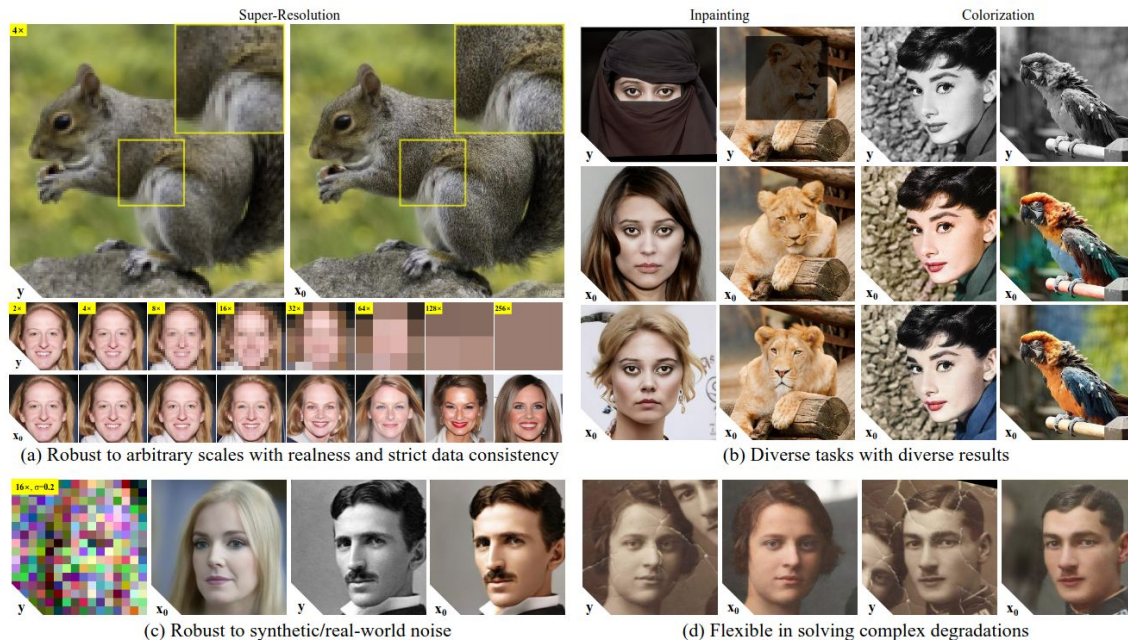


Image credit: Zero-Shot Image Restoration Using Denoising Diffusion Null-Space Model <https://arxiv.org/pdf/2212.00490>



# Generative Modeling: A primer

---

Given a dataset  $\mathcal{D} = \{(x, \ell)\}$ , of **samples**  $x$  conditioned on **labels**  $\ell$ , we want to learn a model  $p_{\theta}(x|\ell)$  where pairs from  $\mathcal{D}$  have high likelihood.

What if we just maximize  $\log p_{\theta}(x|\ell)$ ?

This yields a **variational autoencoder**.

Kingma, D. P., & Welling, M. (2013). Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*.

# Generative Modeling: VAE (2013)

---

Consider an **encoder**  $g_\theta(x, \ell) \rightarrow z$  and a **decoder**  $h_\theta(z, \ell) \rightarrow x$

Let  $q_\theta(z|x, \ell)$  arising from encoding  $x$  from the data

distribution and  $p_\theta(x|z, \ell)$  from decoding with  $z \sim \mathcal{N}(0, I)$

$$\begin{aligned}\log p_\theta(x|\ell) &= \log \int p_\theta(x|z, \ell) p(z) dz \\ &\geq \mathbb{E}_{z \sim q_\theta(z|x)} [\log p_\theta(x|z, \ell) + \log p(z) - \log q_\theta(z|x, \ell)] \\ &= \underbrace{-\text{D}_{\text{KL}}(q_\theta(z|x, \ell) || p(z))}_{\text{Match Prior over } z} + \underbrace{\mathbb{E}_{z \sim q_\theta(z|x)} [\log p_\theta(x|z, \ell)]}_{\text{"Autoencode"}}$$

Jensen's Inequality

Kingma, D. P., & Welling, M. (2013). Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*.

# Variational Autoencoder (2013)

---

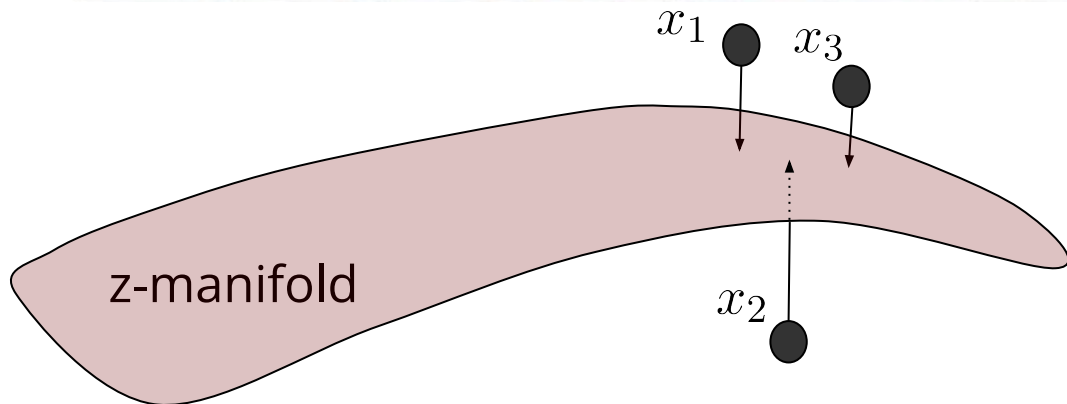
$$\log p_{\theta}(x|\ell) \geq -D_{\text{KL}}(q_{\theta}(z|x, \ell) || p(z)) + \mathbb{E}_{z \sim q_{\theta}(z|x)} [\log p_{\theta}(x|z, \ell)]$$

Choosing  $p(z) = \mathcal{N}(z; 0, I)$  and  $p_{\theta}(x|z, \ell) = \mathcal{N}(x; h_{\theta}(z, \ell), \sigma_1 I)$ ,  
 $q_{\theta}(z|x, \ell) = \mathcal{N}(z; g_{\theta}(x, \ell), \sigma_2 I)$  we can simplify:

$$\log p_{\theta}(x|\ell) \geq \underbrace{-\frac{1}{2} \|g_{\theta}(x, \ell)\|^2}_{\text{"Regularize" to prior}} - \underbrace{\frac{1}{2\sigma_1^2} \mathbb{E}_{\epsilon \sim \mathcal{N}(0, I)} [\|h_{\theta}(g_{\theta}(x, \ell) + \sigma_2 \epsilon, \ell) - x\|^2]}_{\text{Autoencoding loss}} + C$$

# Variational Autoencoder (2013)

1. Generations are **blurry** and don't capture fine-grain details



$$\mathbb{E}_{\epsilon \sim \mathcal{N}(0, I)} [\|h_{\theta}(g_{\theta}(x, \ell) + \sigma_2 \epsilon, \ell) - x\|^2]$$

Decoder averages over all  $x$  which map to the same  $z$ !

Use a more powerful decoder?

# Variational Autoencoder (2013)

---

2. VAEs suffer from **posterior collapse** where the latent becomes **uninformative**, i.e.  $q_{\theta}(z|x) = p(z)$

$$\begin{aligned}\log p_{\theta}(x|\ell) &= \log \int p_{\theta}(x|z, \ell) p(z) dz \\ &\geq -D_{\text{KL}}(q_{\theta}(z|x, \ell) || p(z)) + \mathbb{E}_{z \sim q_{\theta}(z|x)} [\log p_{\theta}(x|z, \ell)]\end{aligned}$$

**The loss allows the decoder  $p_{\theta}(x|z, \ell)$  to capture the entire distribution without “looking” at  $z$ .**

**More powerful models make this problem worse!**

# Generative Adversarial Networks (2015)

---

**Lesson Learned: Do not use a bound on the data likelihood—optimize the distribution itself.**

Minimize the **Total Variation Distance** between the modeled distribution and data distribution

$$TV(p_\theta, p^\star) = \int |p_\theta(x|\ell) - p^\star(x|\ell)| dx$$

# Generative Adversarial Networks (2015)

---

Use **an alternative formulation of the TV distance**:

$$\begin{aligned}TV(p_\theta, p^\star) &= \int |p_\theta(x|\ell) - p^\star(x|\ell)| dx \\ &= \sup_{f, |f| \leq 1} E_{p_\theta}[f(x)] - E_{p^\star}[f(x)]\end{aligned}$$

Putting it together: A **generator**  $G_\theta(z|\ell)$  which maps noise to data.

And a **Discriminator**  $D_\psi(x, \ell) \in [-1, 1]$  which classifies an input as generated or real.

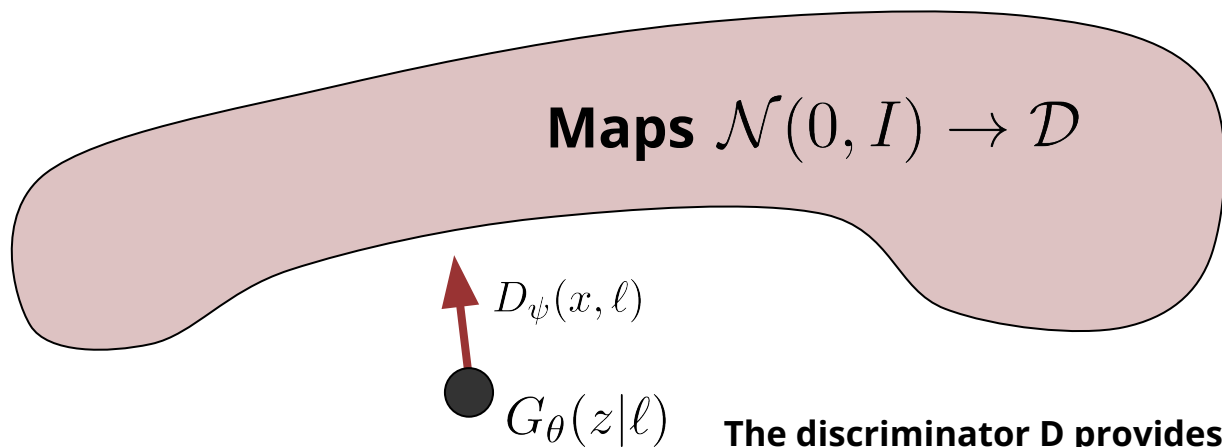
$$L(x, \ell) = D_\psi(x, \ell) - D_\psi(G_\theta(z, \ell), \ell)$$



# Generative Adversarial Networks (2015)

---

$$\min_{\theta} \max_{\psi} \mathbb{E}_{x, \ell} [D_{\psi}(x, \ell) - D_{\psi}(G_{\theta}(z, \ell), \ell)]$$

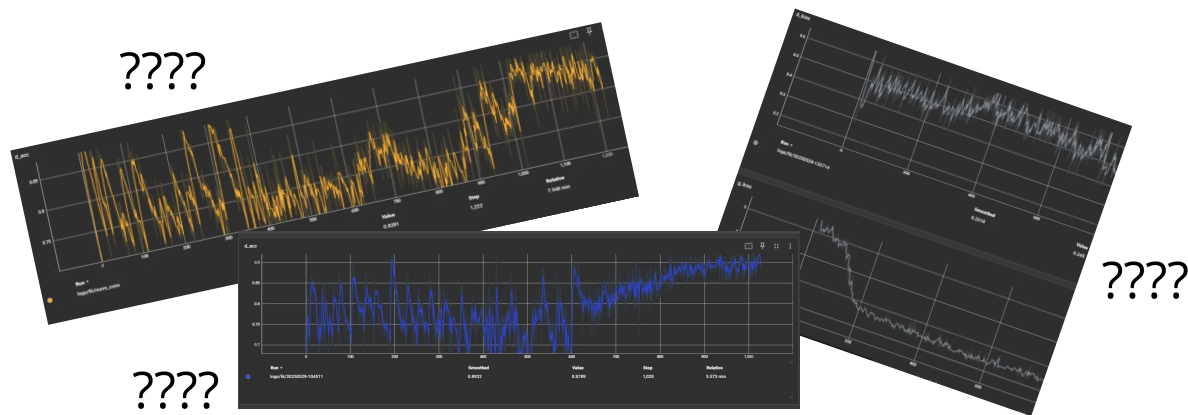


**The discriminator D provides information for how to update G to better map noise to data.**

# Generative Adversarial Networks (2015)

GANs are **hard to train** due to their min-max nature

1. Careful tuning of generator vs discriminator
2. Many, many, many hacks.



SOTA performance though!

| method                        | params | NFE   | FID   |
|-------------------------------|--------|-------|-------|
| <i>GANs</i>                   |        |       |       |
| BigGAN [5]                    | 112M   | 1     | 6.95  |
| GigaGAN [21]                  | 569M   | 1     | 3.45  |
| StyleGAN-XL [40]              | 166M   | 1     | 2.30  |
| <i>autoregressive/masking</i> |        |       |       |
| AR w/ VQGAN [10]              | 227M   | 1024  | 26.52 |
| MaskGIT [6]                   | 227M   | 8     | 6.18  |
| VAR-d30 [47]                  | 2B     | 10×2  | 1.92  |
| MAR-H [27]                    | 943M   | 256×2 | 1.55  |
| <i>diffusion/flow</i>         |        |       |       |
| ADM [8]                       | 554M   | 250×2 | 10.94 |
| LDM-4-G [37]                  | 400M   | 250×2 | 3.60  |
| SimDiff [20]                  | 2B     | 512×2 | 2.77  |
| DiT-XL/2 [34]                 | 675M   | 250×2 | 2.27  |
| SiT-XL/2 [33]                 | 675M   | 250×2 | 2.06  |
| SiT-XL/2+REPA [51]            | 675M   | 250×2 | 1.42  |



Massachusetts Institute of Technology

6.S183 IAP 2026: A practical introduction to diffusion models

# Enter: Diffusion (2019/2020)

---

Two foundational papers:

1. Score-based generative modeling through stochastic differential equations (2019, Song et. al.)
2. Denoising diffusion probabilistic models (2020, Ho et al.)

Key Idea: Fix a **specific** map from noise  $\rightarrow$  data in the form of a **denoising process**.

**No approximations, no min/max or discriminator.**

---

# Course outline

---

1. Introduction (Daniel)
2. Interpretations / perspectives (Daniel)
3. Conditioning and guidance (Cole)
4. Score distillation and additional topics (Artem)
5. Additional applications (Chenyang)
6. Generalization and conclusion (Chris)



# Course Staff

---



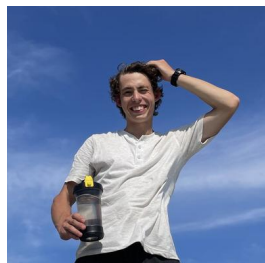
Chenyang Yuan



Cole Becker



Daniel Pfrommer



Artem Lukoianov



Christopher Scarvelis



# Course logistics

---

**Lectures:** M/W/F 10am-11am, Jan 5 -16, 32-144

**Webpage:** <https://practical-diffusion.org/>

All course materials will be posted here

**Piazza:** <https://piazza.com/class/mjehkp2ai4j2vk/>

**Office hours:** Thu 1/8 and Tue 1/13 (Location TBD)

**Email:** [6.s183-diffusion-instructors@mit.edu](mailto:6.s183-diffusion-instructors@mit.edu)

**Mailing list:** <https://mailman.mit.edu/mailman/listinfo/6.s183-diffusion>

# Course grading

---

Grading will be P/D/F, 18 points and above required to pass

## **Attendance/participation** (6 points)

- Complete short survey sent after each lecture to receive credit

## **Problem Sets** (12 points)

- Two problem sets (Pset 1 released today, due Friday 1/10)
- Graded on completion

## **Mini Project** (4 points)

- Open-ended project (see website for project ideas)
- Report due end of class

# What are diffusion models?

Inspired by diffusion processes  
(Brownian motion)

Forward process adds noise to data

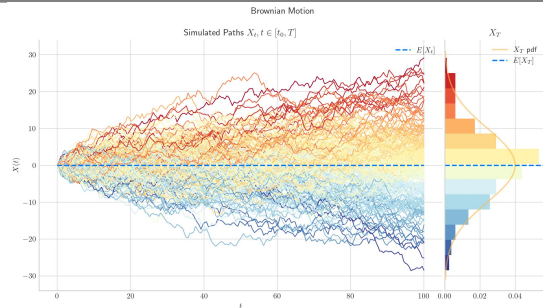


Image credit:  
[https://quantgirluk.github.io/Understanding-Quantitative-Finance/brownian\\_motion.html](https://quantgirluk.github.io/Understanding-Quantitative-Finance/brownian_motion.html)

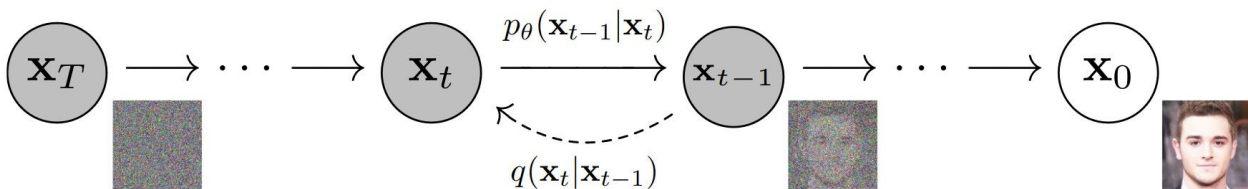


Image credit: Denoising Diffusion Probabilistic Models <https://arxiv.org/pdf/2006.11239>

Diffusion models learn the reverse process to recover data from noise



# Training diffusion models

---

Denoising diffusion models estimate a **noise vector**  $\epsilon \in \mathbb{R}^n$  from a given **noise level**  $\sigma > 0$  and noisy input  $x_\sigma \in \mathbb{R}^n$  such that for some  $x_0$  in the **data manifold**  $\mathcal{K}$ ,

$$x_\sigma \approx x_0 + \sigma \epsilon$$

A **denoiser**  $\epsilon_\theta : \mathbb{R}^n \times \mathbb{R}_+ \rightarrow \mathbb{R}^n$  is learned by minimizing

$$L(\theta) := \mathbf{E}_{x_0, \sigma, \epsilon} \|\epsilon_\theta(x_0 + \sigma \epsilon, \sigma) - \epsilon\|^2$$

$x_0$  is sampled from training data

$\sigma$  is sampled from *training noise schedule* (more on this later)

$\epsilon$  is sampled from  $N(0, I_n)$  (i.i.d. Gaussian)

# Training noise schedules

What levels of noise to use?

In practice, noise levels range from

$$\sigma_{\min} = 0.01 \text{ to } \sigma_{\max} = 100$$

Sampled uniformly from a *noise schedule* (typically 100-1000 discrete noise levels)

$$\sigma \sim \{\sigma_1, \dots, \sigma_{1000}\}$$

Typically follow a log-scale  $\lambda = \log(\sigma)$

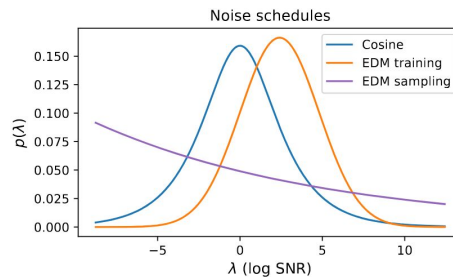
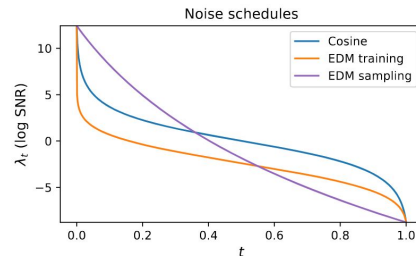
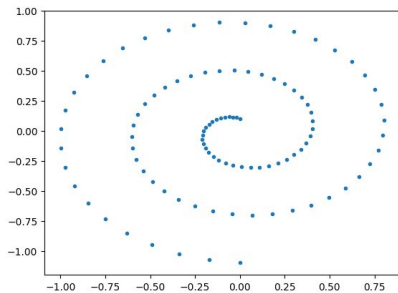


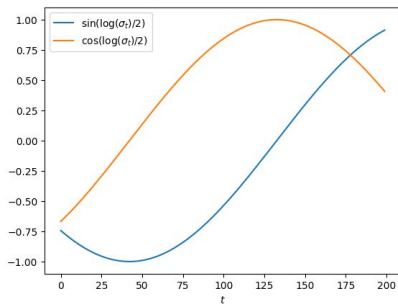
Image credit: Understanding Diffusion Objectives as the ELBO with Simple Data Augmentation <https://arxiv.org/pdf/2303.00848>

# Toy model

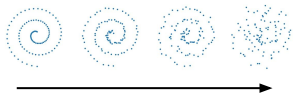
2D spiral dataset



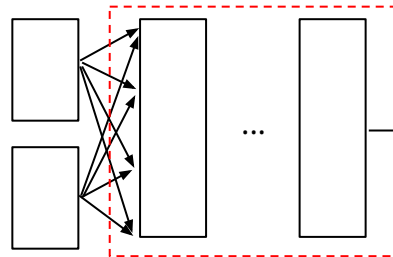
2D noise-level embedding



Add noise



Fully Connected MLP



Predict Noise

Noise-level  
embedding

```
def get_sigma_embeds(sigma):
    sigma = sigma.unsqueeze(1)
    return torch.cat([torch.sin(torch.log(sigma)/2),
                     torch.cos(torch.log(sigma)/2)], dim=1)

class TimeInputMLP(nn.Module):
    def __init__(self, dim, hidden_dims):
        super().__init__()
        layers = []
        for in_dim, out_dim in pairwise((dim + 2,) + hidden_dims):
            layers.extend([nn.Linear(in_dim, out_dim), nn.GELU()])
        layers.append(nn.Linear(hidden_dims[-1], dim))
        self.net = nn.Sequential(*layers)
        self.input_dims = (dim,)

    def rand_input(self, batchsize):
        return torch.randn((batchsize,) + self.input_dims)

    def forward(self, x, sigma):
        sigma_embeds = get_sigma_embeds(sigma)  # shape: b x 2
        nn_input = torch.cat([x, sigma_embeds], dim=1)  # shape: b x (dim + 2)
        return self.net(nn_input)

model = TimeInputMLP(dim=2, hidden_dims=(16, 128, 128, 128, 16))
```

# Commonly used model architectures

## Convolutional U-Nets

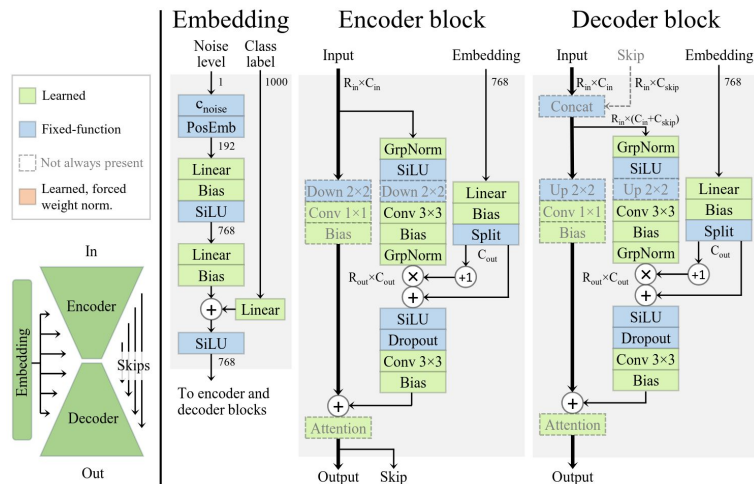


Image credit: Analyzing and Improving the Training Dynamics of Diffusion Models  
<https://arxiv.org/pdf/2312.02696>

## Patch-wise Transformers

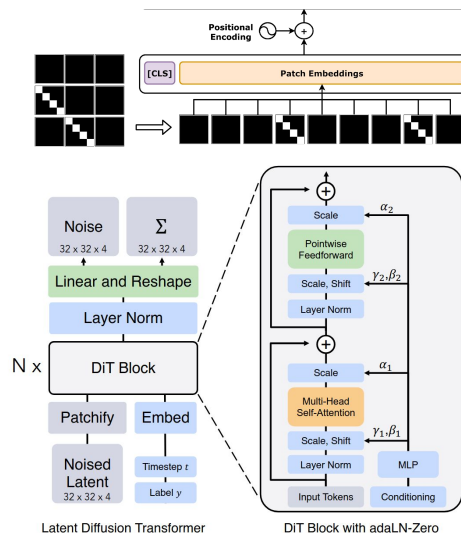
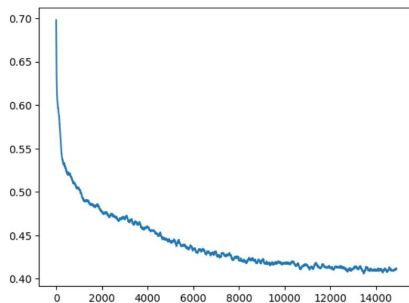


Image credit: Scalable Diffusion Models with Transformers  
<https://arxiv.org/pdf/2212.09748>



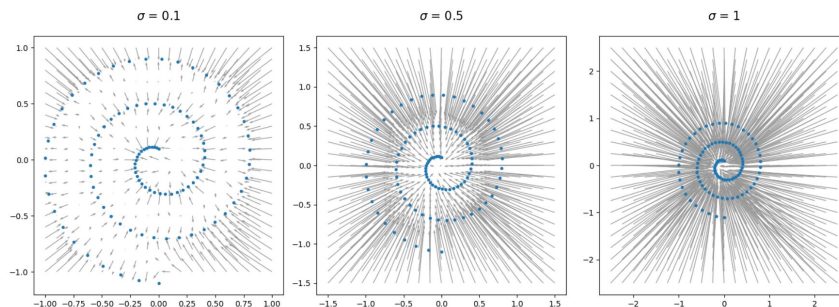
# Training the toy model

```
def training_loop(loader : DataLoader,
                  model : nn.Module,
                  schedule: Schedule,
                  epochs : int = 10000):
    optimizer = torch.optim.Adam(model.parameters())
    for _ in range(epochs):
        for x0 in loader:
            optimizer.zero_grad()
            sigma, eps = generate_train_sample(x0, schedule)
            eps_hat = model(x0 + sigma * eps, sigma)
            loss = nn.MSELoss()(eps_hat, eps)
            optimizer.backward(loss)
            optimizer.step()
```



Training loss over 15000 epochs, smoothed with moving average

The learned denoiser  $\epsilon_\theta(x, \sigma)$  can be visualized as a vector field parameterized by the noise level  $\sigma$ , by plotting  $x - \sigma\epsilon_\theta(x, \sigma)$  for different  $x$  and levels of  $\sigma$ .



Plot of predicted  $\hat{x}_0 = x - \sigma\epsilon_\theta(x, \sigma)$  for different  $x$  and  $\sigma$

The model predicts expected  $x_0$  given  $x$  and  $\sigma$

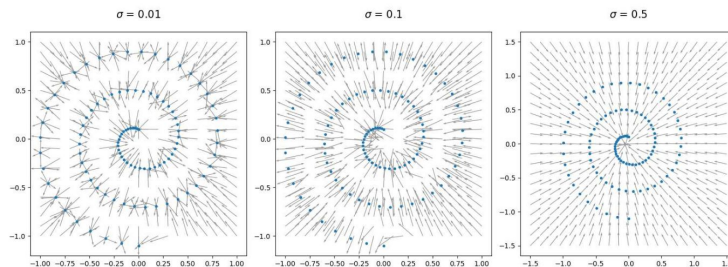
# What did the model learn?

The *ideal denoiser* is the minimizer of training loss, a function of data distribution  $\mathcal{K}$  and noise level  $\sigma$

$$\epsilon^* := \arg \min_{\epsilon_\theta} \mathbf{E}_{x_0, \sigma, \epsilon} \|\epsilon_\theta(x_0 + \sigma \epsilon, \sigma) - \epsilon\|^2$$

For finite  $\mathcal{K}$ , there is a closed-form solution:

$$\epsilon^*(x_\sigma, \sigma) = \frac{\sum_{x_0 \in \mathcal{K}} (x_\sigma - x_0) \exp(-\|x_\sigma - x_0\|^2 / 2\sigma^2)}{\sigma \sum_{x_0 \in \mathcal{K}} \exp(-\|x_\sigma - x_0\|^2 / 2\sigma^2)}$$



Plot of direction of  $\epsilon^*(x, \sigma)$  for different  $x$  and  $\sigma$

# What did the model learn?

The *distance function*  $\text{dist}_{\mathcal{K}} : \mathbb{R}^n \rightarrow \mathbb{R}$  to a set  $\mathcal{K} \subseteq \mathbb{R}^n$ , is defined via

$$\text{dist}_{\mathcal{K}}(x) := \inf\{\|x - x_0\| : x_0 \in \mathcal{K}\}$$

Intuitively,  $\text{proj}_{\mathcal{K}}(x) - x$  is the direction of steepest descent (i.e. neg. gradient) of  $\text{dist}_{\mathcal{K}}(x)$ .  $\nabla \frac{1}{2} \text{dist}_{\mathcal{K}}(x)^2 = x - \text{proj}_{\mathcal{K}}(x)$

Distance function is not differentiable everywhere,  $\nabla \text{dist}_{\mathcal{K}}(x)$  is not continuous, thus hard to learn!

Solution:  $\sigma$ -smoothed distance function

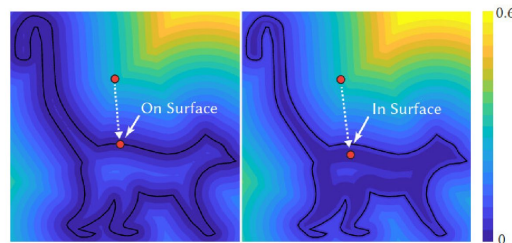
$$\text{dist}_{\mathcal{K}}(x, \sigma) := \underset{x_0 \in \mathcal{K}}{\text{softmax}}_{\sigma^2} \|x_0 - x\|^2 = -\sigma^2 \log \left( \sum_{x_0 \in \mathcal{K}} \exp \left( -\frac{\|x_0 - x\|^2}{2\sigma^2} \right) \right)$$

## Theorem

For all  $\sigma > 0$  and  $x \in \mathbb{R}^n$ , we have

$$\frac{1}{2} \nabla_x \text{dist}_{\mathcal{K}}^2(x, \sigma) = \sigma \epsilon^*(x, \sigma).$$

Denoiser model learns the gradient of a  $\sigma$ -smoothed distance function



Smoothed distance function has continuous gradients

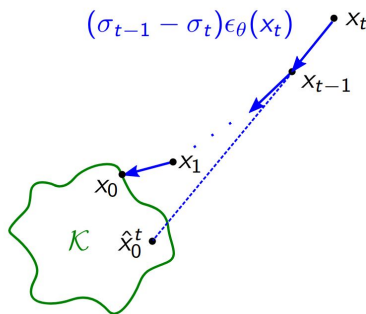
Image credit: Fast Evaluation of Smooth Distance Constraints on Co-Dimensional Geometry <https://arxiv.org/pdf/2108.10480>

# Sampling from diffusion models

---

Given noisy  $x_\sigma$  and noise level  $\sigma$ , the learned denoiser  $\epsilon_\theta(x_\sigma, \sigma)$  estimates

$$x_0 \approx \hat{x}_0(x_\sigma, \sigma) := x_\sigma - \sigma \epsilon_\theta(x_\sigma, \sigma).$$



Sampling algorithms (e.g. DDIM) construct a sequence  $\hat{x}_0^t := \hat{x}_0(x_t, \sigma_t)$  of estimates from a sequence of points  $x_t$  using the update:

$$x_{t-1} = x_t + (\sigma_{t-1} - \sigma_t)\epsilon_\theta(x_t, \sigma_t)$$



# Implementing the sampler

---

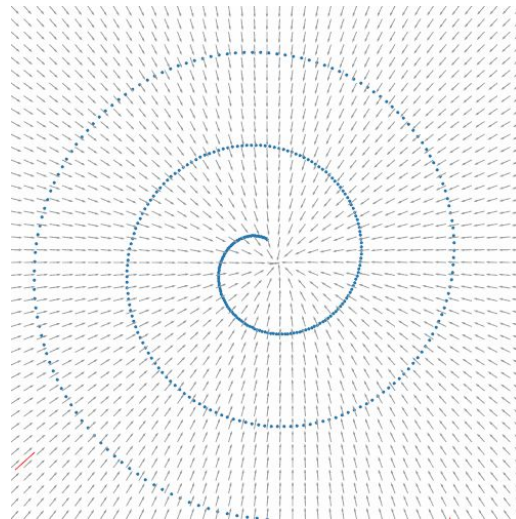
## Algorithm 1 DDIM sampler

---

```
for  $t = N, \dots, 1$  do  
     $x_{t-1} \leftarrow x_t + (\sigma_{t-1} - \sigma_t)\epsilon_\theta(x_t, \sigma_t)$   
return  $x_0$ 
```

---

```
@torch.no_grad()  
def samples_ddim(model : torch.nn.Module,  
                 sigmas : torch.FloatTensor,  
                 xt : Optional[torch.FloatTensor] = None,  
                 batchsize : int = 1):  
    model.eval()  
    accelerator = Accelerator()  
    xt = model.rand_input(batchsize).to(accelerator.device) * sigmas[0]  
    for i, (sig, sig_prev) in enumerate(pairwise(sigmas)):  
        xt = xt - (sig - sig_prev) * model.predict_eps(xt, sig.to(xt))  
    return xt
```



# Probabilistic sampling

Deterministic (DDIM) update:

$$x_{t-1} = x_t + (\sigma_{t-1} - \sigma_t)\epsilon_\theta(x_t, \sigma_t)$$

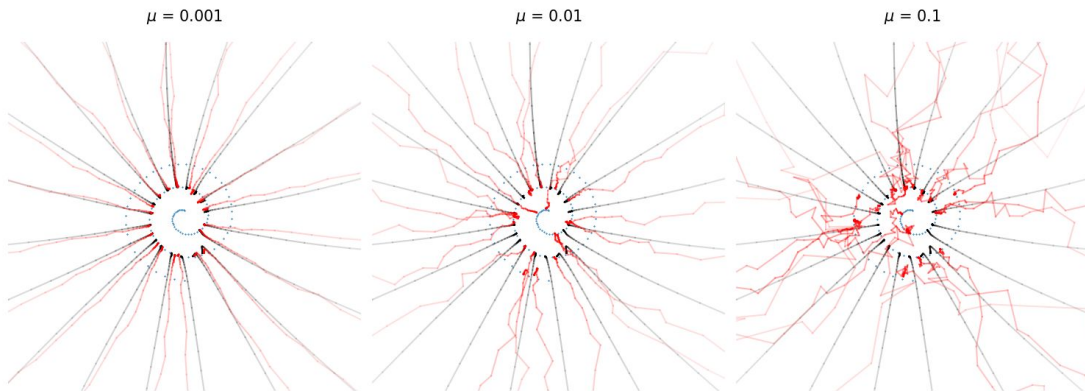
Probabilistic (DDPM) update:

$$x_{t-1} = x_t + (\sigma_{t'} - \sigma_t)\epsilon_\theta(x_t, \sigma_t) + \eta w_t$$

Where  $w_t \sim \mathcal{N}(0, I)$ ,  $\sigma_{t-1} = \sigma_t^\mu \sigma_{t'}^{1-\mu}$  and  $\eta = \sqrt{\sigma_{t-1}^2 - \sigma_{t'}^2}$ .

(Matches norm of update in expectation if  $\mathbb{E} \|w_t\|^2 = \|\epsilon_\theta(x_t, \sigma_t)\|^2$ )

Note:  $0 \leq \mu < 1$  and  $\sigma_{t'} < \sigma_{t-1} < \sigma_t$



# Additional reading/resources

---

Blogpost: <https://chenyang.co/diffusion.html>

Code: <https://github.com/yuanchenyang/smaldiffusion/>

Elucidating the Design Space of Diffusion-Based Generative Models  
<https://arxiv.org/abs/2206.00364>

Analyzing and Improving the Training Dynamics of Diffusion Models  
<https://arxiv.org/abs/2312.02696>

See course website for more readings

**Pset 1 released today, due Friday 1/10**